

Группа М3209

К работе допущен_

Студент Кузнецова А.А

Работа выполнена_

Преподаватель Писарева Ю.И.

Отчет принят

Рабочий протокол и отчет по моделированию № 1

Задание 3 "Лунолет"

1. Условие

"Лунолет"

Инженер-электронщик лунной базы Иванов, тестируя автопилот малого лунного корабля забыл отключить цепь управления двигателем. В результате двигатель включился и корабль взлетел. Потеряв сознание от перегрузки, Иванов очнулся через некоторое время после отключения двигателя и обнаружил, что находится на высоте H_0 и движется со скоростью V_{0y} .

Помогите Иванову благополучно посадить корабль на Луну. Для этого необходимо рассчитать на какой высоте необходимо включить двигатель, чтобы совершить безопасную посадку. Вертикальная посадочная скорость не должна превышать 3 м/с

Используемые константы:

1. Масса аппарата M .
2. Масса топлива m .
3. Ускорение свободного падения g_L на Луне.
4. Предельная перегрузка при маневрах $a_{\max}$.
5. Скорость истечения продуктов сгорания из реактивного двигателя V_p .

Исходные данные (в скобках значения констант). Ускорение силы тяжести на Луне 1.62 м/с^2 ($g_L = 1.62$). Масса корабля 2000 кг, плюс пилот в скафандре 150 кг ($M = 2150$).

Двигатель работает на керосине с жидким кислородом — скорость истечения продуктов сгорания 3660 м/с ($V_p = 3660$.) Начальные скорость и высота определены в варианте задания. В баках 150 кг топлива и окислителя ($m = 150$). Расход топлива двигателем составляет 15 кг/с.

Для расчетов скорости воспользуйтесь уравнением Мещерского.

Упрощения модели:

- Рассматривается движение только по вертикали.
- Поверхность Луны считается плоской

Входные данные: согласно варианту

Построить графики зависимости вертикальной скорости V_y , ускорения a_y , и высоты H от времени.

Вывести значение вертикальной скорости на высоте 0.

2. Цель работы:

Целью данного задания является моделирование вертикальной посадки

лунолета с учётом запаса топлива, допустимой вертикальной скорости и других параметров.

3. Рабочие формулы и исходные данные.

1. Функция для расчёта скорости по уравнению Мещерского

Для расчёта изменения скорости с учетом расхода топлива используется **уравнение Мещерского**:

$$\Delta V = V_p \ln \left(\frac{M_0}{M_f} \right)$$

где:

- ΔV — изменение скорости аппарата,
- V_p — скорость истечения продуктов сгорания из двигателя (в м/с),
- M_0 — начальная масса аппарата (включая топливо),
- M_f — масса аппарата после расхода топлива.

Масса аппарата M_f в момент времени t рассчитывается как:

$$M_f = M_0 - \dot{m} \cdot t$$

где:

- M_0 — начальная масса аппарата,
- $\dot{m}$ — расход топлива (в кг/с),
- t — время (в секундах).

Итак, скорость аппарата $V(t)$ на момент времени t с учетом гравитационного ускорения g_L на Луне будет вычисляться по формуле:

$$V(t) = V_0 + V_p \ln \left(\frac{M_0}{M_0 - \dot{m} \cdot t} \right) - g_L \cdot t$$

где:

- V_0 — начальная вертикальная скорость аппарата.

Ускорение аппарата $a(t)$ на момент времени t можно выразить как:

$$a(t) = \frac{F_{\text{двиг}}}{M_f} - g_L$$

где:

- $M_f = M_0 - \dot{m} \cdot t$ — масса аппарата на момент времени t ,
- g_L — ускорение свободного падения на Луне (1.62 м/с²).

$$F_{\text{двиг}} = \dot{m} \cdot V_p$$

для $\dot{m}$ - расход топлива и V_p - скорость истечения продуктов сгорания, получаем

$$a(t) = \frac{\dot{m} \cdot V_p}{M_0 - \dot{m} \cdot t} - g_L$$

Исходные данные:

Высота H_0 , м	Вертикальная скорость V_{0y} , м/с
2500	14

4. Код программы (Python)

```
[1]: import math
import matplotlib.pyplot as plt
```

```
[2]: 1# Константы
M = 2150 # Масса корабля с пилотом (кг)
m_fuel = 150 # Масса топлива (кг)
g_L = 1.62 # Ускорение свободного падения на Луне (м/с^2)
Vp = 3660 # Скорость истечения продуктов сгорания (м/с)
dot_m = 15 # Расход топлива (кг/с)
V_safe = 3 # Максимальная безопасная посадочная скорость (м/с)
```

```
[3]: # Блок 2: Начальные условия (редактируйте для расчётов)
H0 = 2500 # Начальная высота (м)
V0 = 14 # Начальная скорость (м/с, отрицательное значение для снижения)
```

```
[4]: # Функция для расчёта скорости по уравнению Мещерского
def velocity(t, M_full, M_empty, V0):
    current_mass = M_full - dot_m * t
    if current_mass < M_empty:
        return None # Топливо закончилось
    return V0 + Vp * (math.log(M_full / current_mass)) - g_L * t
```

```
[5]: # Функция для расчёта ускорения
def acceleration(t, M_full, M_empty):
    current_mass = M_full - dot_m * t
    if current_mass < M_empty:
        return None # Топливо закончилось
    return (Vp * dot_m / current_mass) - g_L
```

```
[6]: # Блок 4: Основная функция расчёта
```

```
def calculate_landing():
    """
    Рассчитывает высоту включения двигателя и проверяет условия безопасной посадки.
    """
    # Проверка на корректность начальных данных
    if H0 <= 0:
        print("Ошибка: начальная высота должна быть больше 0.")
        return
    if V0 < 0:
        print("Ошибка: начальная скорость должна быть положительной для данной задачи.")
        return

    # Масса корабля без топлива
    M_empty = M

    # Полная масса корабля
    M_full = M + m_fuel

    # Время работы двигателя
    burn_time = m_fuel / dot_m

    # Численный поиск времени включения двигателя
    t_on = 0
    H = H0
    dt = 0.01 # Шаг интегрирования (с)

    time_data = []
    height_data = []
    velocity_data = []
    acceleration_data = []
    final_velocity = None

    while H > 0:
        if t_on > burn_time:
            print("Топливо закончилось до посадки!")
            break

        v = velocity(t_on, M_full, M_empty, V0)
        a = acceleration(t_on, M_full, M_empty)

        if v is None or a is None:
            print("Ошибка: топлива недостаточно для дальнейшего расчёта.")
            break

        # Условие включения двигателя для безопасной посадки
        if abs(v) <= V_safe and H > 0:
            break

        # Проверки на физические ограничения
        if H < 0:
            H = 0
            final_velocity = v
            break
```

```

if t_on > 1000: # Ограничение времени для предотвращения зацикливания
    print("Ошибка: расчёты занимают слишком много времени, проверьте входные данные.")
    break

H -= v * dt
t_on += dt

if H <= 0:
    final_velocity = v

# Сохранение данных для графиков
time_data.append(t_on)
height_data.append(max(H, 0)) # Гарантируем, что высота не отрицательная
velocity_data.append(v)
acceleration_data.append(a)

```

```

print(f"Высота включения двигателя: {max(H, 0):.2f} м")
print(f"Оставшееся топливо: {max(0, m_fuel - dot_m * t_on):.2f} кг")
if final_velocity is not None:
    print(f"Вертикальная скорость на высоте 0: {final_velocity:.2f} м/с")

# Построение графиков
plt.figure(figsize=(15, 10))

# График высоты
plt.subplot(3, 1, 1)
plt.plot(time_data, height_data, label="Высота (м)")
plt.xlabel("Время (с)")
plt.ylabel("Высота (м)")
plt.title("Изменение высоты")
plt.legend()
plt.grid(True)

# График скорости
plt.subplot(3, 1, 2)
plt.plot(time_data, velocity_data, label="Скорость (м/с)", color="orange")
plt.xlabel("Время (с)")
plt.ylabel("Скорость (м/с)")
plt.title("Изменение скорости")
plt.legend()
plt.grid(True)

```

```

# График ускорения
plt.subplot(3, 1, 3)
plt.plot(time_data, acceleration_data, label="Ускорение (м/с^2)", color="green")
plt.xlabel("Время (с)")
plt.ylabel("Ускорение (м/с^2)")
plt.title("Изменение ускорения")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```

[7]:

```

# Блок 5: Запуск расчётов
if __name__ == "__main__":
    calculate_landing()

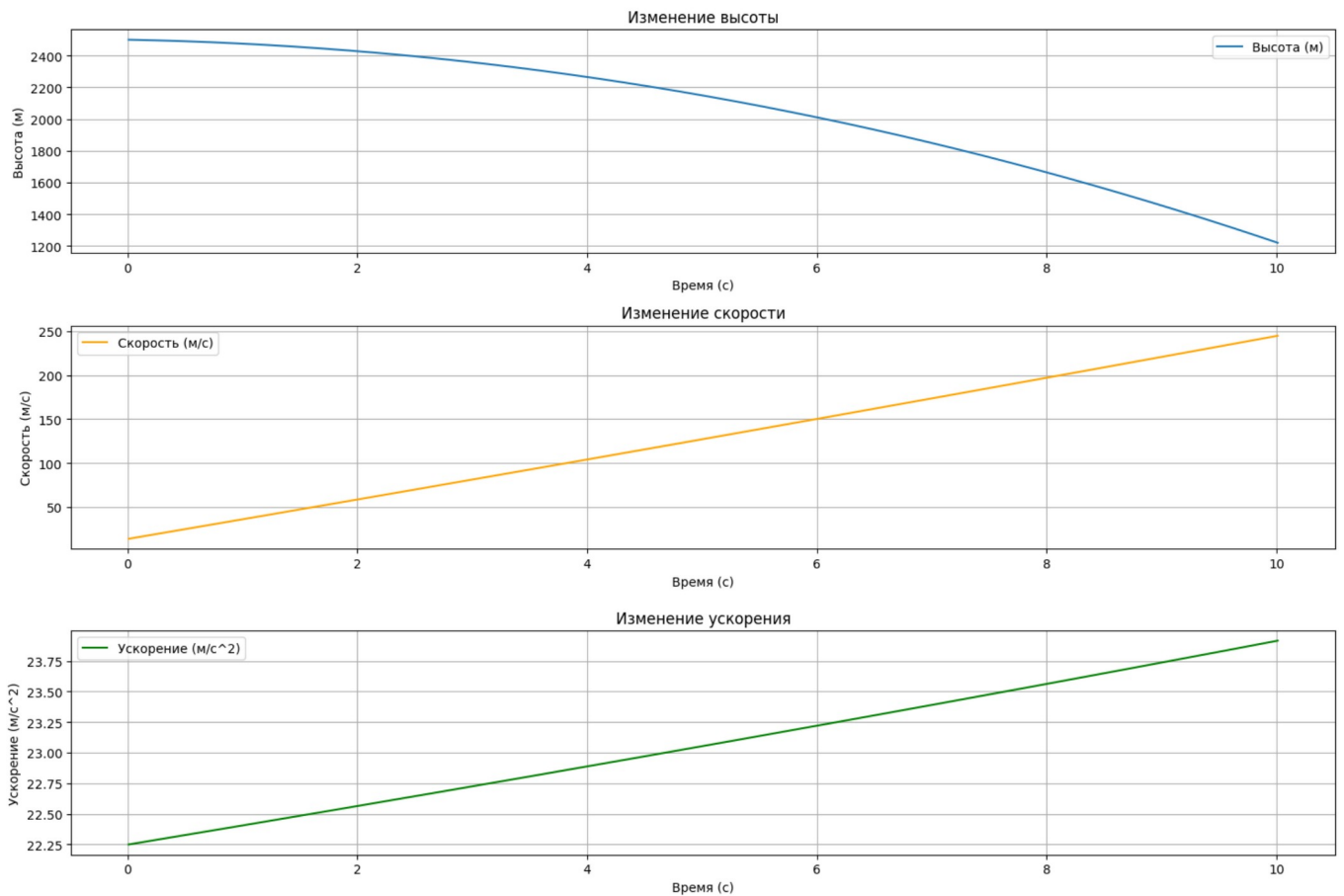
```

5. Результаты выполнения

Топливо закончилось до посадки!

Высота включения двигателя: 1219.40 м

Оставшееся топливо: 0.00 кг



6. Окончательные результаты.

При заданных условиях посадить лунолет не получится